

企业知识库问答机器人

基于 RAG 架构的智能知识库问答系统

课程：云计算与大数据应用开发

演讲讲义（共 13 页 PPT · 预计 15 分钟）

2026 年 6 月

开场白 & 封面

📄 屏幕上展示：PPT 封面 — 项目名称、团队信息

🕒 1 分钟

🗣️ 演讲词：

各位老师、同学大家好。我们小组今天带来的项目是“**企业知识库问答机器人**”——一个基于 RAG 架构的智能问答系统。

这个项目面向企业办公场景：公司内部有大量的制度文件、培训材料、报销流程等文档，员工查找起来非常耗时。我们的系统把这些文档“喂”给 AI，员工直接用自然语言提问，几秒钟就能得到准确回答。相当于给企业建了一个会说话的“知识大脑”。

接下来我会从项目背景、技术架构、功能演示、实现细节和部署方案五个方面展开介绍。

项目背景 — 为什么要做这个系统

📺 屏幕上展示：三个卡片 — 企业痛点、知识沉淀、AI 赋能

🕒 1.5 分钟

🗣️ 演讲词：

先来看**企业痛点**。在座的如果有过实习经历可能会深有体会——新人入职问"报销怎么走"、"IT 设备怎么申请"、"考勤制度是什么"，常常要到处翻找文件，或者反复问 HR 和行政，效率很低。大公司可能有专门的 helpdesk，但中小企业往往没有这样的资源。

第二是**知识沉淀**的问题。公司其实并不缺文档——制度、流程、培训材料都有，但它们只是堆在共享文件夹里，像一本没有目录的字典。隐性知识没有被有效地组织和利用起来。

第三是**AI 赋能**。2023 年以来大语言模型发展非常快，但通用大模型并不了解企业内部的具体内容，而且还存在"幻觉"问题——会编造不存在的答案。RAG 架构正好解决了这个矛盾：检索先找到相关文档，再由大模型基于真实文档作答，做到"言之有据"。

项目目标 — 系统要实现的四个核心能力

📺 屏幕上展示：四个编号目标 — 多格式知识库、智能问答、多端访问、多模态扩展

🕒 1 分钟

🗣️ 演讲词：

我们的系统设定了四个目标。

第一，多格式知识库。不仅要支持传统的 PDF、Word、Excel，还扩展到了图片和视频。也就是说企业里的培训视频、产线照片也能被“读懂”并纳入知识库。

第二，智能问答。用户用自然语言提问，系统检索相关知识后交给 DeepSeek 大模型生成回答，并且每条回答都会标注引用来源和相关度分数——不是“AI 说的”，而是“文档里写的”。

第三，多端访问。既有 Streamlit 做的 Web 界面给普通用户使用，也有 FastAPI 提供的 RESTful 接口方便其他系统集成。同时还支持 Docker 容器化部署。

第四，多模态扩展。图片通过 DeepSeek 的视觉模型理解内容，视频则通过 OpenAI Whisper 做语音识别转文字，实现全格式覆盖。

技术架构 — RAG 全链路

📺 屏幕上展示：RAG 流程图（文档上传→文本提取→向量嵌入→ChromaDB语义检索→LLM生成）+ 技术栈表格

🕒 1.5 分钟

🗣️ 演讲词：

这是整个系统的**技术架构图**，也是 RAG 的完整链路。

流程分为两个阶段。第一阶段是**知识入库**：文档上传后经过多模态处理（文本直接提取、图片走视觉模型理解、视频走 Whisper 语音识别），然后用 LangChain 的文本分割器切成合适大小的片段，再调用 Embedding 模型把每个片段转成 512 维的向量，存入 ChromaDB 向量数据库。

第二阶段是**问答检索**：用户提问后，同样把问题转成向量，在 ChromaDB 中做语义检索——用的是余弦相似度，找到最相关的 Top-K 个片段。然后把这些片段作为"参考资料"拼接到 prompt 里，交给 DeepSeek 大模型生成最终回答。

下方的技术栈表格列出了我们每一层选用的技术。大模型用 DeepSeek，向量库用 ChromaDB，Embedding 默认用 BAAI 的 bge-small 中文模型（本地免费方案），前端用 Streamlit，API 用 FastAPI。整套技术栈都是开源且业界主流的。

功能展示 — 智能问答界面

 **屏幕上展示：** 问答界面截图 — 用户输入"IT设备怎么申请"，AI 回答并附带引用来源 🕒 1 分钟

演讲词：

这张截图展示的是我们的**核心功能——问答界面**。

大家可以看到，用户在底部输入框输入了"IT设备怎么申请"这个问题。系统首先去知识库中检索相关的文档片段，找到了 IT 设备申领流程的相关内容，然后由 DeepSeek 大模型基于这些片段生成回答。

注意这里有一个重要设计：回答下方有一个**"引用来源"**的折叠区域，展开后可以看到每条答案依据了哪些文档、相关度分数有多高。这让回答变得**可追溯、可验证**——不是大模型随口说的，而是有据可查的。对企业场景来说，这个可信度非常重要。

功能展示 — 知识库管理

 **屏幕上展示：** 知识库管理界面截图 — 文件上传区域、文档列表、统计指标

 1 分钟

演讲词：

这一页展示的是**知识库管理**功能。

系统支持 8 种文件格式的直接上传：PDF、Word、Markdown、TXT、Excel、JPG 图片、PNG 图片、MP4 视频。点击上传或者拖拽文件即可，也可以输入一个本地文件夹路径进行批量导入。

上传后系统会自动完成文本提取、分块、向量化和存储的全流程，进度条会实时反馈处理进度。文档列表中可以看到每份文档的名称、类型和片段数量，支持单个删除或一键清空。

底部的三个统计指标——文档数、切片数、向量库大小——让你对知识库的规模一目了然。

功能展示 — 侧边栏配置与历史管理

🖥️ **屏幕上展示：**侧边栏截图 — API 配置、Embedding 切换、Top-K 调节、历史记录列表

🕒 1 分钟

🗣️ 演讲词：

侧边栏包含了系统的**配置中心**。

第一个亮点是**Embedding API 切换**。系统支持三种向量化方案：本地免费的 BGE 模型（开箱即用）、OpenAI 的 text-embedding-3-small（精度更高但需 API Key）、以及阿里云的 DashScope（国内网络友好）。可以根据预算和精度需求灵活选择。

检索参数也是实时可调的：**Top-K** 控制每次检索返回多少条相关片段，**温度参数**控制回答的创造性，**LLM 模型**可以在 deepseek-chat 和 deepseek-coder 之间切换。

右侧的**历史记录**区域会按时间倒序列出所有问答，点击可展开查看详情，支持导出 CSV 文件方便存档分析。

API 服务 — FastAPI 后端

📺 屏幕上展示：API 端点表格 + 技术亮点卡片

🕒 1 分钟

🗣️ 演讲词：

除了 Web 界面，我们还搭建了**完整的 RESTful API 服务**，使用 FastAPI 框架。

这里有 5 个核心端点：**/chat** 是 RAG 问答接口，输入问题返回答案和引用来源；**/search** 是纯语义检索，不经过 LLM 直接返回相关文档片段；**/upload** 处理文档上传入库；**/stats** 返回知识库统计数据；**/documents** 管理文档列表。

FastAPI 自动生成了 Swagger 文档（访问 /docs 即可看到），所有接口都可以直接在浏览器中交互式测试。API 层还配置了 CORS 跨域支持，方便移动端或前端框架调用。

这一层的意义在于：**系统不仅是一个独立的 Web 应用，还是一个可以被集成的服务**。比如企业已有的 OA 系统、企业微信机器人，都可以通过 API 接入我们的问答能力。

核心实现 — RAG 检索增强生成

📺 屏幕上展示：四个流程步骤 + 右侧 Python 代码示例

⌚ 1.5 分钟

🗣️ 演讲词：

这一页深入讲一下 RAG 的**四个关键实现环节**。

第一，文档预处理。我们使用 LangChain 的 RecursiveCharacterTextSplitter，chunk_size 设为 500 个字符，overlap 设 50。为什么要有 overlap？因为如果一个知识点恰好被切在边界上，overlap 可以保证相邻 chunks 之间有部分重叠，不会丢失上下文。我们还针对中文优化了分隔符——用句号、换行作为优先切分点。

第二，向量化。默认使用 BAAI 的 bge-small-zh-v1.5 模型，产生 512 维向量。这个模型在中文语义理解上表现很好，而且完全免费本地运行。右侧代码展示了检索流程：调用 retriever.retrieve() 返回三元组（文本、元数据、相似度分数）。

第三，语义检索。ChromaDB 使用余弦距离度量相似度。我们将距离转换为 0 到 1 之间的分数——分数越接近 1 越相关。

第四，生成回答。把检索到的上下文拼接起来，加上系统提示词（要求基于资料回答、不编造），调用 DeepSeek API 生成最终答案。我们还保存了最近 10 轮对话历史，支持多轮追问。

多模态 RAG — 图片 + 视频处理

📺 屏幕上展示：两条处理流程图 — 图片 RAG (5 步) 和视频 RAG (5 步)

🕒 1 分钟

🗣️ 演讲词:

这是本项目的**亮点之一——多模态支持**。传统 RAG 只能处理纯文本，但我们扩展到了图片和视频。

图片 RAG 流程：上传的图片先用 PIL 压缩到 1024 像素以内（控制传输大小），然后转成 Base64 编码，发送给 DeepSeek 的视觉模型进行理解。视觉模型会输出对图片内容的文字描述，这段描述就代表图片进入了向量库。用户搜索时就能命中相关图片。

视频 RAG 流程：上传的 MP4 视频先通过 MoviePy 提取音频轨道（WAV 格式），然后调用 OpenAI Whisper 进行语音识别。Whisper 对中文的识别效果相当不错，把音频转成中文文字后，这段文字就纳入了知识库。

需要注意的一点是：如果视觉模型 API 不可用（比如接口不支持 image_url 格式），系统会自动降级——提取图片的尺寸、格式等元数据作为备用描述，确保图片仍然可被检索到。这是一种**优雅降级**的设计。

容器化部署 — Docker Compose

📄 屏幕上展示：docker-compose.yml 代码 + 部署架构说明卡片

🕒 1 分钟

🗣️ 演讲词:

部署方面，我们采用了 **Docker Compose 多服务编排**。

docker-compose.yml 定义了两个服务：**streamlit** 运行 Web 界面在 8501 端口，**api** 运行 FastAPI 在 8000 端口。两个服务使用同一个 Dockerfile 构建，共享 data 数据卷——这样向量库和上传的文档在容器重启后不会丢失。

右侧列出了部署架构的四个关键设计：多阶段构建减小镜像体积、数据卷持久化、环境变量通过 .env 注入保证安全、restart: unless-stopped 实现进程崩溃后的自愈重启。

只需要一行命令 **docker compose up -d** 即可同时启动两个服务。这完全符合云计算平台的一键部署要求。

技术难点与解决方案

📺 屏幕上展示：5 个难点卡片 — 问题（黄色）+ 解决方案（绿色）+ 技术标签 ⌚ 1 分钟

🗣️ 演讲词：

在开发过程中，我们遇到了几个关键的技术问题，这里简单分享一下解决方案。

第一个，ChromaDB 的元数据 None 崩溃。某些情况下向量库返回的 metadata 是 None，直接调用 .get() 会抛出 AttributeError。我们在检索器中增加了 None 检查，用空字典兜底。

第二个，Streamlit 缓存导致知识库不更新。VectorStore 被 @st.cache_resource 装饰后不会重新读取 ChromaDB，用户上传了文档却查不到。我们移除了这个装饰器，改为每次查询实时连接数据库，同时在侧边栏提供了手动清除缓存的按钮。

第三，中文长文本分割精度问题。简单按字符数分割会切断句子。我们用 LangChain 的 RecursiveCharacterTextSplitter 配合中文分隔符解决。

第四，多进程 SQLite 锁冲突。Streamlit 和 API 同时访问 ChromaDB 可能产生锁冲突。我们的方案是：读操作并发、写操作统一由 API 处理。

第五，VL 模型降级。当视觉模型 API 不可用时，自动提取图片元数据作为备用描述，保证系统不崩溃。

总结与展望

📄 屏幕上展示：6 个创新点清单 + 团队成员分工

🕒 1 分钟

🗣️ 演讲词：

最后做一个总结。我们实现了**六个关键成果**：

完整 RAG 链路——从文档入库到智能问答全流程打通；多模态支持——文本、图片、视频三种输入形式全部覆盖；多端访问——Streamlit Web 界面 + FastAPI + Docker 一键部署；灵活 Embedding——三种向量化方案可切换；来源追溯——每条回答标注引用文档和相关度分数；工程化实践——容错处理、降级策略、容器化部署。

在技术选型上，我们融合了**大语言模型（AI 技术）**、**向量数据库（云计算基础设施）**和**Docker 容器化（云原生部署）**，这些都紧扣了“云计算与大数据应用开发”的课程主题。

未来可以继续优化的方向包括：接入企业微信/钉钉等即时通讯工具、支持更多文件格式（如邮件、网页）、引入用户反馈机制来持续改进检索质量。

以上就是我们项目的全部内容，谢谢各位老师！请各位提问。

附录 A：关键技术术语解释

术语	解释
RAG	Retrieval Augmented Generation，检索增强生成。在 LLM 生成回答之前，先从外部知识库检索相关文档，将文档内容作为"参考资料"一起输入给模型，提高回答的准确性和可信度。
Embedding（向量嵌入）	将文本映射为高维空间中的向量，语义相近的文本向量距离也相近。本项目使用 512 维向量。
余弦相似度	衡量两个向量方向一致程度的指标。值域 [-1, 1]，越接近 1 表示越相似。ChromaDB 使用余弦距离（1 - 余弦相似度）。
Vector Store	向量数据库，专门存储和检索高维向量。本项目用 ChromaDB，底层是 SQLite。
Chunk（文本分块）	将长文档切分成小段，每段独立做向量化和检索。本项目 chunk_size=500 字符，overlap=50。
Top-K	检索时返回最相似的前 K 个结果。K 越大覆盖面越广，但噪声也越多。默认 K=4。
LLM	Large Language Model，大语言模型。本项目使用 DeepSeek 的 deepseek-chat 模型，兼容 OpenAI API 格式。
VL（视觉模型）	Vision Language Model，能理解图片内容的多模态大模型。本项目用 DeepSeek 的 VL 能力处理上传的图片。
Whisper	OpenAI 开源的语音识别模型，支持多语言（含中文）。本项目用于将视频音频转为文字。
Docker Compose	Docker 的多容器编排工具，用一个 YAML 文件定义和启动多个服务。本项目用它同时管理 Streamlit 和 API 两个服务。

附录 B：常见答辩问题及回答思路

可能的问题	回答思路
-------	------

为什么选 ChromaDB 而不是 Milvus/Pinecone?	ChromaDB 轻量、零配置、嵌入 Python 进程, 适合小规模知识库和课程项目; Milvus 更适合生产级海量向量。可以日后升级。
如果用户上传了敏感文件怎么办?	当前版本是本地部署, 数据不离本地。后续可加权限控制和数据脱敏。
RAG 和微调 (Fine-tuning) 有什么区别?	RAG 不改变模型参数, 通过检索注入外部知识, 灵活且可解释; 微调需要标注数据且知识更新需重新训练。两者互补。
回答质量怎么评估?	引用来源的相关度分数是定量指标; 也可以人工评估回答的准确性、完整性和流畅性。目前以引用来源追溯作为质量保障手段。
视频处理为什么选 Whisper 而不是其他方案?	Whisper 开源、中文识别效果好、可本地运行。缺点是模型较大 (~1.5GB), 首次加载慢。
系统支持多少并发用户?	Streamlit 是单进程模式, 适合小团队使用 (十几人同时)。高并发场景可通过 FastAPI 部署多 worker 实例扩展。
知识库更新后需要重新建索引吗?	不需要。每次上传文档是增量式添加, ChromaDB 自动更新索引。已删除的文档需要调用删除接口。

💡 **演讲提示:** 建议提前打开 Streamlit 和 API 两个窗口备用。在讲解 Slide 5-7 功能展示时可以切换到实际运行的界面做 live demo (如果网络条件允许)。整体演讲控制在 15 分钟以内, 留 3-5 分钟给 Q&A。